Can a green Python release pipeline be trusted?

Soup DevOps internship - Amir - 12 November 2026

Verdict and acceptance status

Can it publish without a human? TestPyPI yes. Real PyPI not yet. Four naive runs passed; the final verifier rejected every fault with the expected failed-consumer error and green source checks. The v0.9.4 release passed all 21 jobs, including OIDC publication and public wheel/sdist read-back. The package is on TestPyPI. Earlier startup failures are retained as infrastructure failures. No production package or broken fixture was published.

Repository: soup-devops-release-lab. The complete evidence and every retained remote attempt are in the evidence ledger.

Four incidents, one weak pipeline

The CLI reads a TOML service config and bundled JSON defaults, validates it, and prints JSON. Fixed version: 0.9.4; isolated faulty fixtures: 0.9.3. Naive CI uses a Linux Python 3.12 editable install, two library tests, and a build. All four fixtures passed naive GitHub Actions runs and built successfully. A separate local reproduction also recorded four naive passes and four successful builds.

Fault	What users see	Hardened consumer that rejects it
Python compatibility	Unconditional tomllib import fails on 3.10	Python 3.10 CLI launch
OS compatibility	Unix-only pwd import prevents Windows startup	Native Windows CLI launch
Missing wheel data	Source has defaults.json; installed wheel does not	Inspect the installed wheel.
Broken entry point	Console command targets missing_main	Invoke the installed executable

Each rejection log contains the intended error signature. The fixed wheel and sdist passed six clean consumers: Linux 3.10, Linux 3.12, and Windows 3.13, two formats each. All 107 tests passed with 96.15% combined package line/branch coverage; adversarial release-guard tests cover artifact and identity failures. The unprivileged Docker runtime launched with network disabled, read-only filesystem, and capabilities dropped.

What changed in the release path

Actions lint/type checks and unit tests accompany one build. The fixed matrix covers Python 3.10-3.14 on Linux, Windows, and macOS; the recorded fixed run passed every leg, including wheel and sdist consumers. Consumers use fresh venvs outside the checkout, install the wheel or rebuild the sdist, verify the import location, launch both entry points, and test valid/invalid configs including a Unicode path. Both distribution identity and SHA-256 hashes are checked. A version tag must equal built metadata and installed CLI output.

Only the publisher gets OIDC permission, after every check. It downloads the exact same-run artifact ID, verifies commit/tag/hashes, and uploads those bytes through the SHA-pinned PyPA action, without checkout or build. Public wheel/sdist hashes matched that artifact and both passed Linux 3.10 read-back. A fresh Windows consumer also downloaded the wheel from TestPyPI's Simple Index and passed. Provenance identity/digest fields were inspected; signatures were not independently verified. Read-back detects a bad publication; it cannot reverse one.

Silent failures closed and what the pipeline still does not catch

Implemented controls close at least three concrete paths: exact transitive hash locks plus one explicit index reduce dependency swaps; fresh installs without restored caches expose resolver errors; immutable artifact IDs and commit/hash checks reject stale bundles; fork PR checks have read-only permissions and no secrets/OIDC; exact tag/metadata matching prevents version drift. Actions and the Docker base are pinned. These controls come first because a passing source test cannot validate the artifact being shipped or which job can publish it. Dependency and cache controls also reduce the chance of approving a build that only works in a reused environment.

Hashes preserve chosen bytes, including malicious ones. Initial dependency selection, unknown vulnerabilities, compromised maintainers/runners, untested architectures, system libraries, user environments, and behavior outside these fixtures remain risks. Attestations prove origin, not correctness. Downstream pip users do not automatically inherit our runtime hash lock. Repository branch/tag protections and independent review remain prerequisites for real PyPI.

GPU release and nightly design

This CLI needs no GPU; the requested design covers a future CUDA component and has not rented hardware. Use Runpod Secure Cloud for on-demand RTX 4090 pods that can be destroyed after each run. The $0.89/hour rate was checked on 2026-11-12. Initially budget one validated software tuple: two monthly releases at 20 test minutes plus 3 startup minutes, and 30 nightlies at 10 plus 3. Total: 436 minutes = $6.47; add 20% retry reserve and $1 small storage/log allowance: $8.76/month. The $8.76 estimate excludes broker and watchdog hosting. Larger compatibility sets require rebudgeting.

Separately, I ran an AI-generated CUDA probe on my own RTX 5090: 65,536 vector-add results matched the CPU reference (exit 0), the no-GPU control failed (exit 1), and a CPU-only timeout returned 124. Test containers were removed. This validates a basic CUDA environment, not GPU CI automation or cloud pricing. See the manual lab report.

Allocate only after CPU gates. An OIDC broker admits protected refs, enforces one pod and a $15 GPU allocation ceiling, and never gives provider credentials to the runner. Separate release/nightly queues protect pending releases. Use 25/15-minute TTLs, an independent minute-by-minute termination watchdog with heartbeat alerts, signed result upload URLs, and reconciliation after ambiguous API responses. Cancel superseded nightlies; retry only transient infrastructure failures once. Slow startup, provider outages and retained storage can defeat a naive cost estimate; the ledger is not a provider hard cap.

What to fix first

With OIDC publication and read-back verified, first protect source, workflow, lock, and tag changes through independent review and account recovery. Then implement a durable release circuit breaker and broaden compatibility coverage. Today a maintainer must manually disable later releases after a publish/read-back failure. Keep the production approval gate until those controls are demonstrated.

Reflection and AI use

I was surprised that the naive checks passed on Linux even though the installed Windows CLI could not start because it imported `pwd`. I was also surprised that the audit flagged pinned pytest (PYSEC-2026-1845), setuptools (PYSEC-2026-3447 and GHSA-xmj3-9gf7-h52w), and wheel (CVE-2026-24049). The pins were corrected and the audit rerun. I'm still concerned that one repository controls both the build and the evidence used to approve it. Hashes identify the bytes that shipped. Deciding whether they should ship still requires independent review.

AI agents generated the implementation. I reviewed the core parts and remained involved throughout development. In Orca, Codex coordinated implementation and checks; OpenCode tightened verifier checks; Antigravity completed an initial review, with its stalled follow-up recorded. Agents and GitHub Actions executed the historical package checks; I ran the separate lab probe. This does not establish personal code authorship, package-test execution, debugging, or independent validation of every result. The disclosure records the current review confirmation and preserves historical tool outcomes. AI-use disclosure.

Sources: PyPI OIDC, GitHub security, Runpod pricing. Detailed threat model, GPU assumptions, disclosure, and reproduction commands are in the repository.